Facultad de Ingeniería de la Universidad de Buenos Aires

Números Amigos

Materia: Teoría de Algoritmos

Alumno: Sebastián Brizuela

Padrón: 105288

1° Cuatrimestre 2026

Índice

Índice **2**
1. Introducción **3**
1.1. Objetivos 3
1.2. Definición del Problema 3
1.3. Alcance y Restricciones 3
2. Supuestos **3**
3. Análisis de complejidad **3**
3.1. Algoritmo Original 3
3.2. Algoritmo Propuesto 4
4. Medición de tiempos **4**
5. Análisis de resultados **5**
6. Conclusión **5**
ANEXO **6**
Algoritmo Original 6
Algoritmo Propuesto 7

1. Introducción

1.1. Objetivos

El propósito de este informe es documentar la reingeniería y optimización de un algoritmo diseñado para hallar números amigos en el rango ***MAX***. Se busca transformar una solución eficaz pero ineficiente en una funcionalmente equivalente con un rendimiento más óptimo.

1.2. Definición del Problema

Se define en el enunciado que dos números son amigos cuando la suma de los divisores propios del primero es igual al segundo, y viceversa. El programa original presenta una demora de **581 segundos** para un ***MAX*** de **100.000**, mostrándose ineficiente para casos con valores mayores.

1.3. Alcance y Restricciones

- **Lenguaje:** El desarrollo debe realizarse estrictamente en Python 3.10.
- **Transparencia:** La optimización no debe alterar el prototipo de la función ***`amigos(MAX)`***.

2. Supuestos

Para garantizar la consistencia de los resultados obtenidos tras la optimización, se establecen los siguientes puntos:

- **Inclusión de Números Perfectos:** Se asume que los números perfectos (donde la suma de divisores de ***n*** es igual a ***n***) deben ser informados, manteniendo el comportamiento del algoritmo original provisto.
- **Definición de Divisores:** Se consideran divisores propios a los números menores al número evaluado que lo dividen de forma exacta.
- Las mediciones de tiempo se realizaron utilizando el entorno **Google Colab**. Se deja constancia de que los tiempos de ejecución pueden variar significativamente según el hardware utilizado, siendo notablemente más veloces en ejecuciones locales. Sin embargo, la eficiencia relativa del algoritmo propuesto se mantiene constante independientemente del entorno.

3. Análisis de complejidad

3.1. Algoritmo Original

El código original (*ANEXO: Algoritmo Original*) realiza las siguientes operaciones para cada número ***i*** en el rango ***[0, MAX)***:

- **Primer Bucle (Cálculo de *s*):** Itera desde **1** hasta ***i*** para encontrar los divisores de ***i***. En el peor de los casos, realiza ***n*** iteraciones.

- **Segundo Bucle (Cálculo de s_2):** Itera desde ***1*** hasta ***s*** para encontrar los divisores de la suma obtenida. Dado que ***s*** puede ser igual a ***n***, este bucle también tiende a realizar ***n*** iteraciones.

Dado que ambos procesos están anidados (para cada ***i*** se ejecutan las búsquedas de divisores), además, considerando que tanto el número evaluado (***i***) como la suma de sus divisores (***s***) en el peor de los casos se asemejan a ***n***, podríamos representarlos en la ecuación con ***n***:

$$T(n) = \sum_{i=1}^{n} (n + n) \approx O(n^2)$$

3.2. Algoritmo Propuesto

En esta nueva versión (*ANEXO: Algoritmo Propuesto*), se aplicaron dos optimizaciones reducen el tiempo de ejecución:

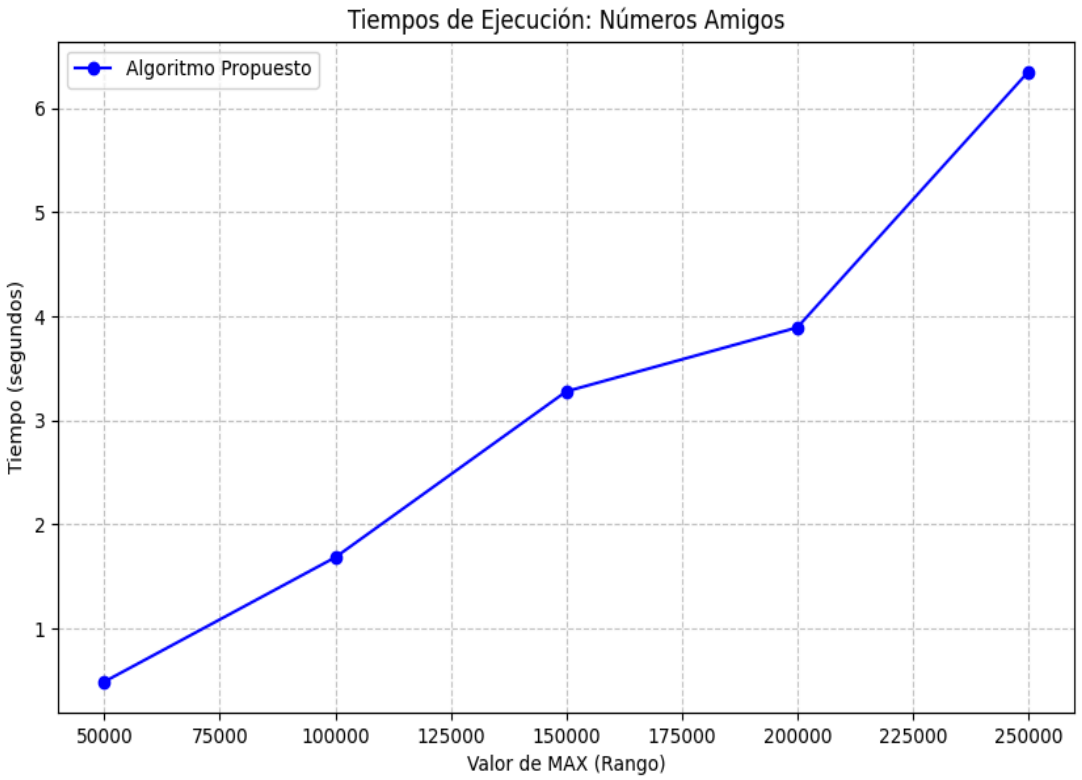
- Se cambió el límite de los bucles internos. En lugar de buscar divisores hasta ***n-1***, se utiliza la raíz cuadrada del número actual ($n^{0.5}$).
- Se eliminó la búsqueda de divisores en casos innecesarios. Por ejemplo, en los casos en que la suma de divisores del primer número daba por resultado un número inferior, ya que probablemente ese par se encontró anteriormente al analizar el número inferior.

Dado que ambos procesos están anidados (para cada ***numero_actual*** se ejecutan las búsquedas de divisores), además, considerando que tanto el número evaluado (***numero_actual***) como la suma de sus divisores (***suma_de_divisores_1***) en el peor de los casos se asemejan a ***n***, podríamos representarlos en la ecuación con ***n***:

$$T(n) = \sum_{i=1}^{n} (\sqrt{n} + \sqrt{n}) \approx O(n \cdot \sqrt{n})$$

4. Medición de tiempos

Rango (MAX)	Tiempo [s]
50000	0.4870
100000	1.6828
150000	3.2780
200000	3.8927
250000	6.3505



5. Análisis de resultados

A partir de los datos obtenidos en la **Sección 4**, se puede observar que comparando el tiempo establecido por el algoritmo original (**581 segundos**) y el tiempo establecido por el algoritmo propuesto (**1.6828 segundos**) para el intervalo de ***1 - 100000***, el nuevo algoritmo redujo del tiempo de ejecución significativamente, validando la eficiencia de la reingeniería.

6. Conclusión

A través de la reingeniería del código original, se demostró que el algoritmo propuesto es significativamente más eficiente, reduciendo los tiempos de ejecución para el mismo volumen de datos.

También se observa que la eficiencia podría incrementarse aún más mediante una refinación de las condiciones de borde. Por ejemplo, al considerar que los **números perfectos** no son estrictamente números amigos (ya que la definición de "amigos" suele implicar dos números distintos), se podrían excluir de la búsqueda para ahorrar iteraciones adicionales. Sin embargo, para este trabajo se optó por incluirlos siguiendo estrictamente la definición y el comportamiento del código propuesto en el enunciado, garantizando así la integridad de los resultados esperados por la cátedra.

ANEXO

Algoritmo Original

```
import time

def amigos(MAX):

  t1=time.time()
  for i in range(MAX):
    s=0
    for j in range (1,i-1+1):
      if i%j==0:
        s+=j

    s2=0
    for k in range (1,s-1):
      if s%k==0:
        s2+=k

    if i==s2:
      print(i,s)

  t2=time.time()
  print(t2-t1)
```

Algoritmo Propuesto

```
import time

def amigos(MAX):
  inicio_cronometro = time.time()
  for numero_actual in range(MAX):
      suma_divisores_1 = 0
      raiz_1 = int(numero_actual**0.5)

      for divisor in range(1, raiz_1 + 1):
          if numero_actual % divisor == 0:
              suma_divisores_1 += divisor
              if numero_actual // divisor != divisor and numero_actual // divisor != numero_actual:
                  suma_divisores_1 += numero_actual // divisor

      if suma_divisores_1 >= numero_actual:
            suma_divisores_2 = 0
            raiz_2 = int(suma_divisores_1**0.5)

            for divisor_2 in range(1, raiz_2 + 1):
                if suma_divisores_1 % divisor_2 == 0:
                    suma_divisores_2 += divisor_2
                    if suma_divisores_1 // divisor_2 != divisor_2 and suma_divisores_1 // divisor_2 !=
suma_divisores_1:
                        suma_divisores_2 += suma_divisores_1 // divisor_2

            if suma_divisores_2 == numero_actual:
                print(numero_actual, suma_divisores_1)

  fin_cronometro = time.time()
  print(fin_cronometro - inicio_cronometro)
```